
Programming with RDDs

This chapter introduces Spark’s core abstraction for working with data, the resilient distributed dataset (RDD). An RDD is simply a distributed collection of elements. In Spark all work is expressed as either creating new RDDs, transforming existing RDDs, or calling operations on RDDs to compute a result. Under the hood, Spark automatically distributes the data contained in RDDs across your cluster and parallelizes the operations you perform on them.

Both data scientists and engineers should read this chapter, as RDDs are the core concept in Spark. We highly recommend that you try some of these examples in an interactive shell (see “[Introduction to Spark’s Python and Scala Shells](#)” on page 11). In addition, all code in this chapter is available in the book’s [GitHub repository](#).

RDD Basics

An RDD in Spark is simply an immutable distributed collection of objects. Each RDD is split into multiple *partitions*, which may be computed on different nodes of the cluster. RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes.

Users create RDDs in two ways: by loading an external dataset, or by distributing a collection of objects (e.g., a list or set) in their driver program. We have already seen loading a text file as an RDD of strings using `SparkContext.textFile()`, as shown in [Example 3-1](#).

Example 3-1. Creating an RDD of strings with `textFile()` in Python

```
>>> lines = sc.textFile("README.md")
```

Once created, RDDs offer two types of operations: *transformations* and *actions*. *Transformations* construct a new RDD from a previous one. For example, one common transformation is filtering data that matches a predicate. In our text file example, we can use this to create a new RDD holding just the strings that contain the word *Python*, as shown in [Example 3-2](#).

Example 3-2. Calling the filter() transformation

```
>>> pythonLines = lines.filter(lambda line: "Python" in line)
```

Actions, on the other hand, compute a result based on an RDD, and either return it to the driver program or save it to an external storage system (e.g., HDFS). One example of an action we called earlier is `first()`, which returns the first element in an RDD and is demonstrated in [Example 3-3](#).

Example 3-3. Calling the first() action

```
>>> pythonLines.first()
u'## Interactive Python Shell'
```

Transformations and actions are different because of the way Spark computes RDDs. Although you can define new RDDs any time, Spark computes them only in a *lazy* fashion—that is, the first time they are used in an action. This approach might seem unusual at first, but makes a lot of sense when you are working with Big Data. For instance, consider [Example 3-2](#) and [Example 3-3](#), where we defined a text file and then filtered the lines that include *Python*. If Spark were to load and store all the lines in the file as soon as we wrote `lines = sc.textFile(...)`, it would waste a lot of storage space, given that we then immediately filter out many lines. Instead, once Spark sees the whole chain of transformations, it can compute just the data needed for its result. In fact, for the `first()` action, Spark scans the file only until it finds the first matching line; it doesn't even read the whole file.

Finally, Spark's RDDs are by default recomputed each time you run an action on them. If you would like to reuse an RDD in multiple actions, you can ask Spark to *persist* it using `RDD.persist()`. We can ask Spark to persist our data in a number of different places, which will be covered in [Table 3-6](#). After computing it the first time, Spark will store the RDD contents in memory (partitioned across the machines in your cluster), and reuse them in future actions. Persisting RDDs on disk instead of memory is also possible. The behavior of not persisting by default may again seem unusual, but it makes a lot of sense for big datasets: if you will not reuse the RDD,

there's no reason to waste storage space when Spark could instead stream through the data once and just compute the result.¹

In practice, you will often use `persist()` to load a subset of your data into memory and query it repeatedly. For example, if we knew that we wanted to compute multiple results about the README lines that contain *Python*, we could write the script shown in [Example 3-4](#).

Example 3-4. Persisting an RDD in memory

```
>>> pythonLines.persist()

>>> pythonLines.count()
2

>>> pythonLines.first()
u'## Interactive Python Shell'
```

To summarize, every Spark program and shell session will work as follows:

1. Create some input RDDs from external data.
2. Transform them to define new RDDs using transformations like `filter()`.
3. Ask Spark to `persist()` any intermediate RDDs that will need to be reused.
4. Launch actions such as `count()` and `first()` to kick off a parallel computation, which is then optimized and executed by Spark.



`cache()` is the same as calling `persist()` with the default storage level.

In the rest of this chapter, we'll go through each of these steps in detail, and cover some of the most common RDD operations in Spark.

Creating RDDs

Spark provides two ways to create RDDs: loading an external dataset and parallelizing a collection in your driver program.

¹ The ability to always recompute an RDD is actually why RDDs are called “resilient.” When a machine holding RDD data fails, Spark uses this ability to recompute the missing partitions, transparent to the user.

The simplest way to create RDDs is to take an existing collection in your program and pass it to SparkContext's `parallelize()` method, as shown in Examples 3-5 through 3-7. This approach is very useful when you are learning Spark, since you can quickly create your own RDDs in the shell and perform operations on them. Keep in mind, however, that outside of prototyping and testing, this is not widely used since it requires that you have your entire dataset in memory on one machine.

Example 3-5. `parallelize()` method in Python

```
lines = sc.parallelize(["pandas", "i like pandas"])
```

Example 3-6. `parallelize()` method in Scala

```
val lines = sc.parallelize(List("pandas", "i like pandas"))
```

Example 3-7. `parallelize()` method in Java

```
JavaRDD<String> lines = sc.parallelize(Arrays.asList("pandas", "i like pandas"));
```

A more common way to create RDDs is to load data from external storage. Loading external datasets is covered in detail in [Chapter 5](#). However, we already saw one method that loads a text file as an RDD of strings, `SparkContext.textFile()`, which is shown in Examples 3-8 through 3-10.

Example 3-8. `textFile()` method in Python

```
lines = sc.textFile("/path/to/README.md")
```

Example 3-9. `textFile()` method in Scala

```
val lines = sc.textFile("/path/to/README.md")
```

Example 3-10. `textFile()` method in Java

```
JavaRDD<String> lines = sc.textFile("/path/to/README.md");
```

RDD Operations

As we've discussed, RDDs support two types of operations: *transformations* and *actions*. Transformations are operations on RDDs that return a new RDD, such as `map()` and `filter()`. Actions are operations that return a result to the driver program or write it to storage, and kick off a computation, such as `count()` and `first()`. Spark treats transformations and actions very differently, so understanding which type of operation you are performing will be important. If you are ever confused

whether a given function is a transformation or an action, you can look at its return type: transformations return RDDs, whereas actions return some other data type.

Transformations

Transformations are operations on RDDs that return a new RDD. As discussed in “[Lazy Evaluation](#)” on page 29, transformed RDDs are computed lazily, only when you use them in an action. Many transformations are *element-wise*; that is, they work on one element at a time; but this is not true for all transformations.

As an example, suppose that we have a logfile, *log.txt*, with a number of messages, and we want to select only the error messages. We can use the `filter()` transformation seen before. This time, though, we’ll show a filter in all three of Spark’s language APIs (Examples 3-11 through 3-13).

Example 3-11. filter() transformation in Python

```
inputRDD = sc.textFile("log.txt")
errorsRDD = inputRDD.filter(lambda x: "error" in x)
```

Example 3-12. filter() transformation in Scala

```
val inputRDD = sc.textFile("log.txt")
val errorsRDD = inputRDD.filter(line => line.contains("error"))
```

Example 3-13. filter() transformation in Java

```
JavaRDD<String> inputRDD = sc.textFile("log.txt");
JavaRDD<String> errorsRDD = inputRDD.filter(
    new Function<String, Boolean>() {
        public Boolean call(String x) { return x.contains("error"); }
    }
);
```

Note that the `filter()` operation does not mutate the existing `inputRDD`. Instead, it returns a pointer to an entirely new RDD. `inputRDD` can still be reused later in the program—for instance, to search for other words. In fact, let’s use `inputRDD` again to search for lines with the word *warning* in them. Then, we’ll use another transformation, `union()`, to print out the number of lines that contained either *error* or *warning*. We show Python in [Example 3-14](#), but the `union()` function is identical in all three languages.

Example 3-14. `union()` transformation in Python

```
errorsRDD = inputRDD.filter(lambda x: "error" in x)
warningsRDD = inputRDD.filter(lambda x: "warning" in x)
badLinesRDD = errorsRDD.union(warningsRDD)
```

`union()` is a bit different than `filter()`, in that it operates on two RDDs instead of one. Transformations can actually operate on any number of input RDDs.



A better way to accomplish the same result as in [Example 3-14](#) would be to simply filter the `inputRDD` once, looking for either *error* or *warning*.

Finally, as you derive new RDDs from each other using transformations, Spark keeps track of the set of dependencies between different RDDs, called the *lineage graph*. It uses this information to compute each RDD on demand and to recover lost data if part of a persistent RDD is lost. [Figure 3-1](#) shows a lineage graph for [Example 3-14](#).

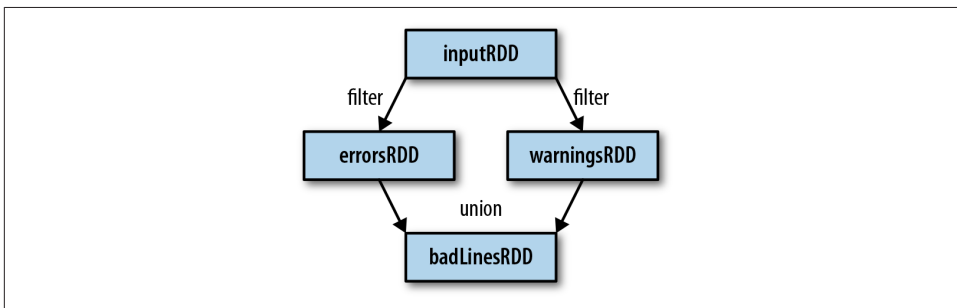


Figure 3-1. RDD lineage graph created during log analysis

Actions

We've seen how to create RDDs from each other with transformations, but at some point, we'll want to actually *do something* with our dataset. Actions are the second type of RDD operation. They are the operations that return a final value to the driver program or write data to an external storage system. Actions force the evaluation of the transformations required for the RDD they were called on, since they need to actually produce output.

Continuing the log example from the previous section, we might want to print out some information about the `badLinesRDD`. To do that, we'll use two actions, `count()`, which returns the count as a number, and `take()`, which collects a number of elements from the RDD, as shown in [Examples 3-15](#) through [3-17](#).

Example 3-15. Python error count using actions

```
print "Input had " + badLinesRDD.count() + " concerning lines"
print "Here are 10 examples:"
for line in badLinesRDD.take(10):
    print line
```

Example 3-16. Scala error count using actions

```
println("Input had " + badLinesRDD.count() + " concerning lines")
println("Here are 10 examples:")
badLinesRDD.take(10).foreach(println)
```

Example 3-17. Java error count using actions

```
System.out.println("Input had " + badLinesRDD.count() + " concerning lines")
System.out.println("Here are 10 examples:")
for (String line: badLinesRDD.take(10)) {
    System.out.println(line);
}
```

In this example, we used `take()` to retrieve a small number of elements in the RDD at the driver program. We then iterate over them locally to print out information at the driver. RDDs also have a `collect()` function to retrieve the entire RDD. This can be useful if your program filters RDDs down to a very small size and you'd like to deal with it locally. Keep in mind that your entire dataset must fit in memory on a single machine to use `collect()` on it, so `collect()` shouldn't be used on large datasets.

In most cases RDDs can't just be `collect()`ed to the driver because they are too large. In these cases, it's common to write data out to a distributed storage system such as HDFS or Amazon S3. You can save the contents of an RDD using the `saveAsTextFile()` action, `saveAsSequenceFile()`, or any of a number of actions for various built-in formats. We will cover the different options for exporting data in [Chapter 5](#).

It is important to note that each time we call a new action, the entire RDD must be computed "from scratch." To avoid this inefficiency, users can *persist* intermediate results, as we will cover in ["Persistence \(Caching\)" on page 44](#).

Lazy Evaluation

As you read earlier, transformations on RDDs are lazily evaluated, meaning that Spark will not begin to execute until it sees an action. This can be somewhat counter-intuitive for new users, but may be familiar for those who have used functional languages such as Haskell or LINQ-like data processing frameworks.

Lazy evaluation means that when we call a transformation on an RDD (for instance, calling `map()`), the operation is not immediately performed. Instead, Spark internally records metadata to indicate that this operation has been requested. Rather than thinking of an RDD as containing specific data, it is best to think of each RDD as consisting of instructions on how to compute the data that we build up through transformations. Loading data into an RDD is lazily evaluated in the same way transformations are. So, when we call `sc.textFile()`, the data is not loaded until it is necessary. As with transformations, the operation (in this case, reading the data) can occur multiple times.



Although transformations are lazy, you can force Spark to execute them at any time by running an action, such as `count()`. This is an easy way to test out just part of your program.

Spark uses lazy evaluation to reduce the number of passes it has to take over our data by grouping operations together. In systems like Hadoop MapReduce, developers often have to spend a lot of time considering how to group together operations to minimize the number of MapReduce passes. In Spark, there is no substantial benefit to writing a single complex map instead of chaining together many simple operations. Thus, users are free to organize their program into smaller, more manageable operations.

Passing Functions to Spark

Most of Spark's transformations, and some of its actions, depend on passing in functions that are used by Spark to compute data. Each of the core languages has a slightly different mechanism for passing functions to Spark.

Python

In Python, we have three options for passing functions into Spark. For shorter functions, we can pass in lambda expressions, as we did in [Example 3-2](#), and as [Example 3-18](#) demonstrates. Alternatively, we can pass in top-level functions, or locally defined functions.

Example 3-18. Passing functions in Python

```
word = rdd.filter(lambda s: "error" in s)
def containsError(s):
    return "error" in s
word = rdd.filter(containsError)
```


One issue to watch out for when passing functions is inadvertently serializing the object containing the function. When you pass a function that is the member of an object, or contains references to fields in an object (e.g., `self.field`), Spark sends the *entire* object to worker nodes, which can be much larger than the bit of information you need (see [Example 3-19](#)). Sometimes this can also cause your program to fail, if your class contains objects that Python can't figure out how to pickle.

Example 3-19. Passing a function with field references (don't do this!)

```
class SearchFunctions(object):
    def __init__(self, query):
        self.query = query
    def isMatch(self, s):
        return self.query in s
    def getMatchesFunctionReference(self, rdd):
        # Problem: references all of "self" in "self.isMatch"
        return rdd.filter(self.isMatch)
    def getMatchesMemberReference(self, rdd):
        # Problem: references all of "self" in "self.query"
        return rdd.filter(lambda x: self.query in x)
```

Instead, just extract the fields you need from your object into a local variable and pass that in, like we do in [Example 3-20](#).

Example 3-20. Python function passing without field references

```
class WordFunctions(object):
    ...
    def getMatchesNoReference(self, rdd):
        # Safe: extract only the field we need into a local variable
        query = self.query
        return rdd.filter(lambda x: query in x)
```

Scala

In Scala, we can pass in functions defined inline, references to methods, or static functions as we do for Scala's other functional APIs. Some other considerations come into play, though—namely that the function we pass and the data referenced in it needs to be serializable (implementing Java's `Serializable` interface). Furthermore, as in Python, passing a method or field of an object includes a reference to that whole object, though this is less obvious because we are not forced to write these references with `self`. As we did with Python in [Example 3-20](#), we can instead extract the fields we need as local variables and avoid needing to pass the whole object containing them, as shown in [Example 3-21](#).

Example 3-21. Scala function passing

```
class SearchFunctions(val query: String) {
  def isMatch(s: String): Boolean = {
    s.contains(query)
  }
  def getMatchesFunctionReference(rdd: RDD[String]): RDD[String] = {
    // Problem: "isMatch" means "this.isMatch", so we pass all of "this"
    rdd.map(isMatch)
  }
  def getMatchesFieldReference(rdd: RDD[String]): RDD[String] = {
    // Problem: "query" means "this.query", so we pass all of "this"
    rdd.map(x => x.split(query))
  }
  def getMatchesNoReference(rdd: RDD[String]): RDD[String] = {
    // Safe: extract just the field we need into a local variable
    val query_ = this.query
    rdd.map(x => x.split(query_))
  }
}
```

If `NotSerializableException` occurs in Scala, a reference to a method or field in a nonserializable class is usually the problem. Note that passing in local serializable variables or functions that are members of a top-level object is always safe.

Java

In Java, functions are specified as objects that implement one of Spark's function interfaces from the `org.apache.spark.api.java.function` package. There are a number of different interfaces based on the return type of the function. We show the most basic function interfaces in [Table 3-1](#), and cover a number of **other function interfaces** for when we need to return special types of data, like key/value data, in “Java” on page 43.

Table 3-1. Standard Java function interfaces

Function name	Method to implement	Usage
<code>Function<T, R></code>	<code>R call(T)</code>	Take in one input and return one output, for use with operations like <code>map()</code> and <code>filter()</code> .
<code>Function2<T1, T2, R></code>	<code>R call(T1, T2)</code>	Take in two inputs and return one output, for use with operations like <code>aggregate()</code> or <code>fold()</code> .
<code>FlatMapFunction<T, R></code>	<code>Iterable<R> call(T)</code>	Take in one input and return zero or more outputs, for use with operations like <code>flatMap()</code> .

We can either define our function classes inline as anonymous inner classes ([Example 3-22](#)), or create a named class ([Example 3-23](#)).

Example 3-22. Java function passing with anonymous inner class

```
RDD<String> errors = lines.filter(new Function<String, Boolean>() {  
    public Boolean call(String x) { return x.contains("error"); }  
});
```

Example 3-23. Java function passing with named class

```
class ContainsError implements Function<String, Boolean>() {  
    public Boolean call(String x) { return x.contains("error"); }  
}  
  
RDD<String> errors = lines.filter(new ContainsError());
```

The style to choose is a personal preference, but we find that top-level named functions are often cleaner for organizing large programs. One other benefit of top-level functions is that you can give them constructor parameters, as shown in [Example 3-24](#).

Example 3-24. Java function class with parameters

```
class Contains implements Function<String, Boolean>() {  
    private String query;  
    public Contains(String query) { this.query = query; }  
    public Boolean call(String x) { return x.contains(query); }  
}  
  
RDD<String> errors = lines.filter(new Contains("error"));
```

In Java 8, you can also use lambda expressions to concisely implement the function interfaces. Since Java 8 is still relatively new as of this writing, our examples use the more verbose syntax for defining classes in previous versions of Java. However, with lambda expressions, our search example would look like [Example 3-25](#).

Example 3-25. Java function passing with lambda expression in Java 8

```
RDD<String> errors = lines.filter(s -> s.contains("error"));
```

If you are interested in using Java 8's lambda expression, refer to [Oracle's documentation](#) and [the Databricks blog post on how to use lambdas with Spark](#).



Both anonymous inner classes and lambda expressions can reference any `final` variables in the method enclosing them, so you can pass these variables to Spark just as in Python and Scala.

Common Transformations and Actions

In this chapter, we tour the most common transformations and actions in Spark. Additional operations are available on RDDs containing certain types of data—for example, statistical functions on RDDs of numbers, and key/value operations such as aggregating data by key on RDDs of key/value pairs. We cover converting between RDD types and these special operations in later sections.

Basic RDDs

We will begin by describing what transformations and actions we can perform on all RDDs regardless of the data.

Element-wise transformations

The two most common transformations you will likely be using are `map()` and `filter()` (see [Figure 3-2](#)). The `map()` transformation takes in a function and applies it to each element in the RDD with the result of the function being the new value of each element in the resulting RDD. The `filter()` transformation takes in a function and returns an RDD that only has elements that pass the `filter()` function.

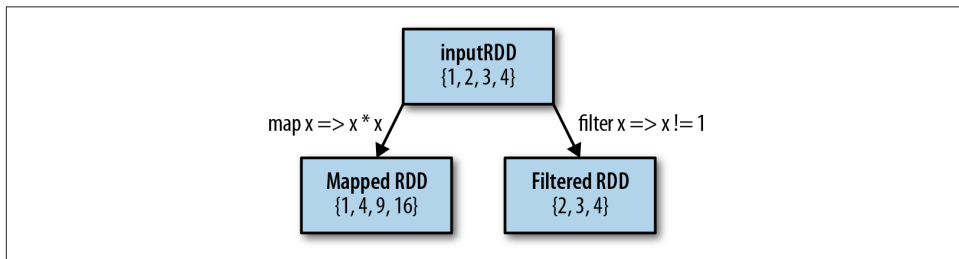


Figure 3-2. Mapped and filtered RDD from an input RDD

We can use `map()` to do any number of things, from fetching the website associated with each URL in our collection to just squaring the numbers. It is useful to note that `map()`'s return type does not have to be the same as its input type, so if we had an RDD `String` and our `map()` function were to parse the strings and return a `Double`, our input RDD type would be `RDD[String]` and the resulting RDD type would be `RDD[Double]`.